Exercise - Block diagrams

Learning Objectives

- Practice capturing architectural ideas on paper for discussion with colleagues
- Experience hierarchy in action and the divide-and-conquer tactics used to handle complexity
- Improve your ability to understand code written by others by drawing diagrams
- Learn tools and tricks that support this process

Preparation

Student Task 1

For ETH users:

- Setup the working directory for this exercise by calling our install script and navigating to the newly created directory. **Important**: make sure you install your exercises on the local hard disks rather than in the accounts home directories (which have a limited disk quota). For this sake, create a directory with your username on the local hard disk (e.g. `/scratch/vlsi2_XXfs26/`), and install the exercise files there. Check out the dedicated Exercise setup page for more information.

```
sh > mkdir -p /scratch/$USER
sh > cd /scratch/$USER
sh > vlsi2 ex02
sh > cd ex02
```

Alternatively, you can directly call the install script for each exercise which you can find under `/home/vlsi2/ex02/install.sh`.

For non-ETH users:

Note: The exercises have been recently updated for the Spring semester 2026. It is possible that some exercise branches on Github require some additional time to be updated accordingly. Please reach out to a maintainer in case of issues.

- Retrieve the files from this exercise from Github:

```
sh > git clone https://github.com/pulp-platform/croc.git
sh > cd croc
sh > git checkout ex02
```

Block Diagrams

Welcome to this week's exercise!

As a digital designer, it is very important that you can properly document your design at different levels of abstraction. This allows other people to take over your work, and it helps your future self get back into the design quickly.

This exercise shows you how to document your design using block diagrams.

Student Task 2

As we just saw, there are several key principles to keep in mind when creating a clear and structured block diagram.

In this task, we will reinforce these concepts with a practical example using a simple drawing tool. You can choose between LibreOffice and Google Slides.

Locate and open the file `task2.odp`. This file contains a poorly formatted block diagram that you will refine using the principles we discussed.

Student Task 3

You will be drawing quite a few block diagrams. Have pen and paper ready!

Always draw your block diagrams with pen and paper first. Afterward, consider using a tool like draw.io (https://app.diagrams.net/) or Inkscape (https://inkscape.org/) to create a clean digital version.

Your project supervisors will be happy to help you with this. People are generally much happier with a good hand-drawn schematic than a poor digital one.

Always start with pen and paper.

The Basics

Let us start by recapitulating the basic digital circuit elements and how to draw them.

Generally, we draw subcircuits as boxes, with the inputs on the left and the outputs on the right. Like parlay, this is more a guideline than a rule. As you will see later, it is often easier to indicate the direction of a port with an arrow and then group ports logically together.

The following figure shows how ports can be annotated with polarity and sensitivity information:

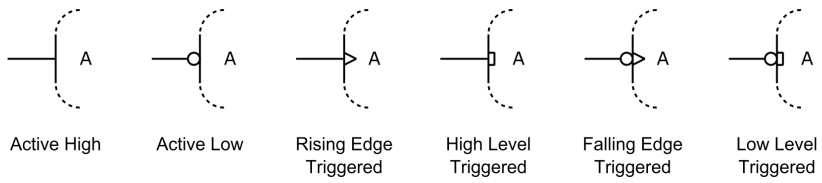

Figure 1: Basic circuit elements

Student Task 4

Quickly think of an example for each of the six cases above. Consider the various types of sequential elements you know.

- Active High:
- Active Low:
- Rising Edge Triggered:
- High Level Triggered:
- Falling Edge Triggered:
- Low Level Triggered:

Always remember to properly label your input and output ports.

In many cases, you do not need to explicitly draw connections for the clock and reset pins unless your design has multiple clocks or resets. Nevertheless, you must include them as ports.

From your bachelor's courses, you should remember the basic digital gates. Let us quickly build a "library" of gates for later reference.

Student Task 5

Draw each of the following gates:
Buffer, AND, OR, Inverter, NAND, NOR, XOR, D-Flip-Flop, XNOR, D-Latch.
Do not forget to label the ports unless there is absolutely no chance of mistaking one for the other.

Show your drawings to an assistant.

This scheme works well for basic gates and storage elements. However, as the subcircuits in your diagram grow in complexity, so will the number of ports leading in and out of them.

If your subcircuit uses multiple handshaking interfaces, for example, it may make more sense to draw related ports close together rather than splitting them into inputs on the left and outputs on the right.
To reduce clutter, multi-bit signals (such as integer values) should be drawn as single thick lines, with an annotation indicating the signal width.

We suggest the following scheme to indicate port directions and annotate signals:

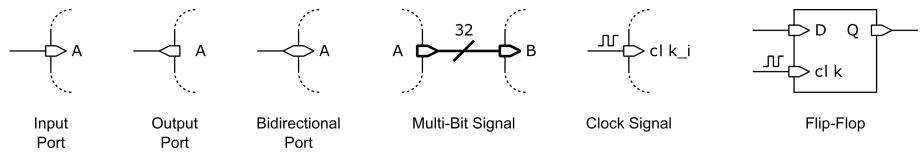

Figure 2: Ports and multi-bit signals

Consider the following entity as an example. As you can see, it has multiple inputs and outputs grouped together by interface and function. It is usually a good idea to group related pins together both in code and in diagrams.

```
module mystery_box (
 input  logic        clk_i,
 input  logic        rst_ni,

 input  logic [41:0] data_i,
 input  logic        valid_i,
 output logic        ready_o,

 input  logic [ 7:0] cfg_addr_i,
 input  logic        cfg_wr_en_i,
 input  logic [31:0] cfg_wr_data_i,
 output logic [31:0] cfg_rd_data_o,

 output logic [41:0] data_o,
 output logic        valid_o,
 input  logic        ready_i
);
/* Mysterious implementation of a box */
endmodule // mystery_box
```

Student Task 6

Draw the block corresponding to the `mystery_box` entity. Do not forget to add the name of the entity.

Show your drawing to an assistant.

Note that circuit design should always start with pencil and a blank sheet of paper. Coming up with architectures and circuit diagrams is where the engineering lies; writing the associated VHDL/SystemVerilog code should mostly be "typing down" what you see in your drawings.

It is also much easier to communicate how a circuit works using a drawing rather than code.

Counters

So far, we have looked at drawing the *outside view* of gates and blocks, and at indicating port directions and functions.

Let us now look at how to document the inner workings of a simple counter. Even though we are now interested in the internals of a block, it is still important to describe how it connects to the outside world.

Consider the following example of a flip-flop:

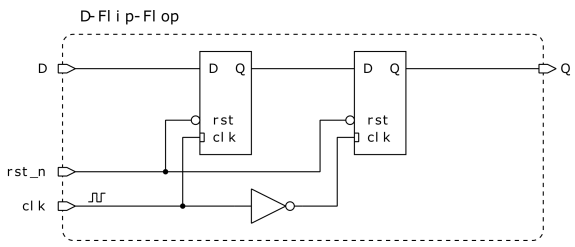

Figure 3: Block diagram of a D-Flip-Flop

If your circuit contains well-known arithmetic operations such as negation, addition, subtraction, multiplication, division, left and right shifts, etc., we suggest drawing them as bubbles with the operation indicated inside.

Use arrows to indicate which signals are the input operands and which signal is the output. For non-commutative operations such as division and shifts, consider explicitly naming the inputs to avoid confusion.

It is also legal to introduce your own symbols, as long as you clearly document their semantics in the diagram.

Consider the following examples:

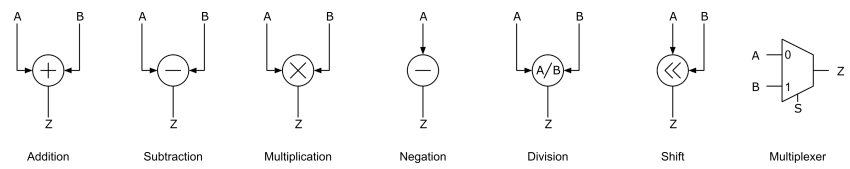

Figure 4: Common arithmetical operations

Let us now look at the following simple counter:

```
module counter (
 input  logic       clk_i,
 input  logic       rst_ni,
 input  logic       enable_i,
 output logic [7:0] count_o
);
logic [7:0] count_d, count_q;

assign count_d = count_q + 1;
assign count_o = count_q;

always_ff @(posedge clk_i or negedge rst_ni) begin
 if (!rst_ni) begin
  count_q <= '0;
 end else if (enable_i) begin
  count_q <= count_d;
 end
end
endmodule // counter
```

Student Task 7

Draw the block diagram of the `counter` entity.

Think about how to draw a multi-bit flip-flop. Consider labeling internal signals as well, not just ports.

Show your drawing to an assistant.

Student Task 8

Now that you have sketched the counter using pen and paper, create a polished version using a vector drawing tool. While you are free to use any vector-based drawing software, this exercise will focus on draw.io.

- Navigate to app.diagrams.net (https://app.diagrams.net) (or open your preferred vector drawing tool).
- Create a new blank diagram.
- Import the custom symbol library:
  - Click `File → Open Library from Device`.
  - Locate and select the custom library file (`.xml` format).

Now you are equipped with a set of pre-made components that will speed up diagram creation and help maintain consistency and clarity.

- Replicate the block diagram from the previous task, ensuring it is neatly organized and visually clear.
- Use proper alignment, consistent font sizes, and clear signal labeling.
- Maintain a clean and structured hierarchy in your diagram.
- Show your drawing to an assistant.

Hierarchical Designs

The complexity of circuits can quickly get out of hand.

The following is a schematic diagram generated by yosys. You can find the corresponding source code in `sourcecode/crc_lfsr.sv`, `sourcecode/crc_hasher.sv`, and `sourcecode/crc_stream.sv`.

As you can see, the design is fairly hierarchical:

- `crc_lfsr` is the forward path of a loadable LFSR
- `crc_hasher` uses the LFSR to compute a hash over input values
- `crc_stream` uses the hasher to append a signature to streams of data bytes

The schematic shown is a flattened version of the design, which is hard to understand. Part of this is due to the noise introduced by the many signal wires, and part is due to the lack of hierarchy.

We will now try to come up with a better schematic for the design.

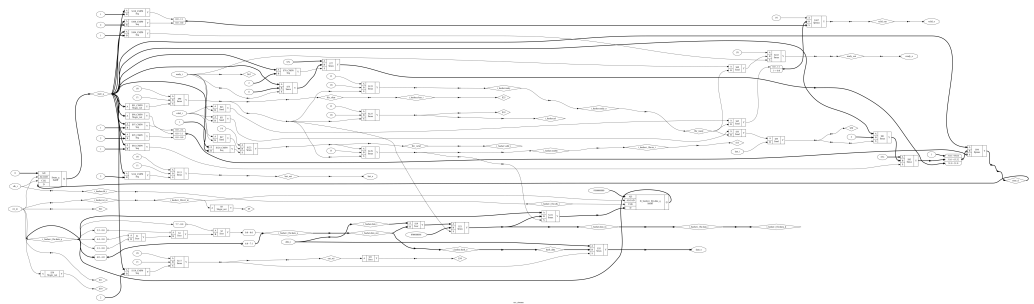

Figure 5: Schematic diagram generated by yosys

An entity instantiation in your code is usually a good indication that you should draw this entity as an opaque box in your block diagram, and provide an additional drawing detailing the internals of the entity.

This also works in the other direction: if it is more comfortable for you to draw something as an opaque box, that is a good indication that the corresponding code might be better off in a separate entity.

These are just hints to get you started; in practice, you must decide case-by-case where to introduce hierarchy and develop your own style.

Student Task 9

Let us start by taking a look at `crc_lfsr` using the text editor of your choice. Answer the following questions:

- How many flip-flops are there?
- What kind of reset do the flip-flops have?
- On which edge do the flip-flops trigger?
- Do the flip-flops have an enable pin?
- What other logic gates are there, and how many?

Student Task 10

Draw the block diagram of the `crc_lfsr` entity.

Consider using multi-bit wires where possible. Do not forget to label your input and output pins, and the entire block.

Show your drawing to an assistant.

Your designs will often contain complex combinational circuitry, for example to compute the next state of a finite state machine or some other non-trivial logic function. These are often very hard to draw.

The *soup* of logic gates they decompose into usually does not help you understand their function and often introduces confusion. For this reason, we suggest abstracting combinational circuitry and FSMs as shown below, and either providing a textual description in the drawing or referring the reader to the source code:

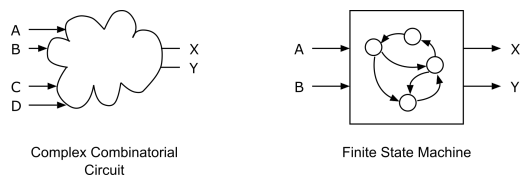

Figure 6: Abstraction of combinatorial circuits and state machines

Student Task 11

Now let us look at `crc_hasher`, which makes use of the LFSR above. Answer the following questions:

- How many instances of `crc_lfsr` are there?
- What kind of gate drives `data_in`?
- What is the purpose of `sel`?

Student Task 12

Draw the block diagram of the `crc_hasher` entity.

We suggest adding the entity name (`crc_*`) above and the instance name (`i_*`) below sub-blocks in your drawing.

Show your drawing to an assistant.

Student Task 13

As the final step, draw the block diagram of the `crc_stream` entity.

Show your drawing to an assistant.

When you abstract away an FSM in future diagrams, consider drawing the state graph of the FSM as a separate diagram. You can refer to signal names in the drawing, making it easy for readers to follow.

Always make good use of hierarchy in your block diagrams, and try to introduce useful layers of abstraction. If you draw a very high-level schematic of a processor, for example, it is probably a bad idea to include every single wire. Instead, abstract away details as black boxes and describe their inner workings in another diagram.

Complex Buses

We can take the concept of abstraction even further.

As your designs grow, you will want to bundle signals into buses that adhere to a well-defined protocol. This can be as simple as a ready/valid handshake where the sender and receiver indicate their ability to produce or consume data.

Doing so allows you to simplify your block diagrams: if a group of signals adheres to a protocol `p`, you may draw them as a single line and annotate which protocol they use.

Consider the following example:

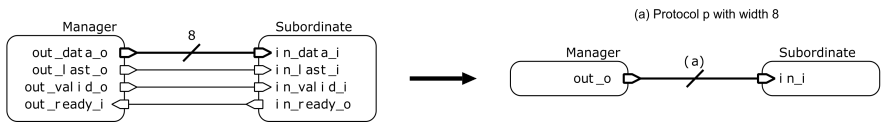

Figure 7: Abstraction of complex buses

Note that a bus will likely contain both input and output signals (for example, the ready and valid signals). However, there is usually a clear direction of information flow.

In the example above, the bus *manager* drives the data and valid signals, while the bus *subordinate* drives the ready signal. Thus, data always flows from the manager to the subordinate.

We suggest using the arrow direction of bus ports to indicate the flow of information.

These protocols are not limited to simple handshakes. Let us have a look at the AXI-Lite [AXI4 specification (http://www.gstitt.ece.ufl.edu/courses/fall15/eel4720_5721/labs/refs/AXI4_specification.pdf)](http://www.gstitt.ece.ufl.edu/courses/fall15/eel4720_5721/labs/refs/AXI4_specification.pdf) interface, which allows subcircuits to exchange data.

It uses addresses to allow the sender to indicate to the receiver which exact piece of information it is interested in. The signal list is as follows:

Signal Name	Width	Manager	Subordinate
AWVALID	1	out	in
AWREADY	1	in	out
AWADDR	*AW*	out	in
AWPROT	1	out	in
WVALID	1	out	in
WREADY	1	in	out
WDATA	*DW*	out	in
WSTRB	*DW/8*	out	in
BVALID	1	in	out
BREADY	1	out	in
BRESP	2	in	out
ARVALID	1	out	in
ARREADY	1	in	out
ARADDR	*AW*	out	in
ARPROT	1	out	in
RVALID	1	in	out
RREADY	1	out	in
RDATA	*DW*	in	out
RRESP	2	in	out

Bus Signals

The exact meaning of each of the 19 signals is not important. However, notice that the interface has two parameters:

- *AW* indicates the width of the address in bits
- *DW* indicates the width of the data in bits

This means that in your block diagrams, you can represent all 19 signals with a single line, as long as you:

- specify that the line is an AXI-Lite bus
- specify *AW*
- specify *DW*
- indicate the manager and subordinate sides with the direction of the port arrows

Student Task 14

Draw the block diagram of the entity `system_on_chip`. You can find the corresponding source code in `sourcecode/system_on_chip.sv`.

Show your drawing to an assistant.

As you have seen, introducing reasonable bus protocols can help keep your diagrams simple and clean while still communicating the key architectural idea.

Multiple Clock Domains

Having multiple clocks notoriously complicates a circuit. Each signal that crosses from one clock domain into another must be treated with great care during synthesis and place-and-route.

It is therefore crucial that the clock domains and the signals crossing between them are easily identifiable. Block diagrams can help with this.

We suggest that you clearly separate circuits running on different clocks and indicate the clock-domain boundary with a line. See the following example:

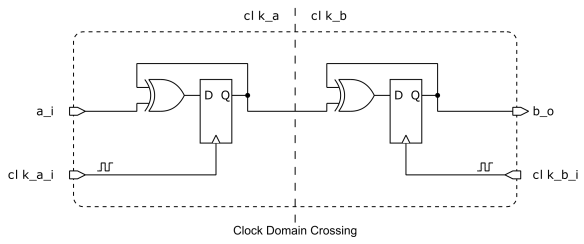

Figure 8: Clock domain crossing

Take a look at the SystemVerilog code in `sourcecode/cdc_2phase.sv`. This is a small circuit that can safely move data between two clock domains.

Your fellow students and the assistants are happy to help if you are struggling to read the SystemVerilog code.

Student Task 15

Draw the block diagram of the module `cdc_2phase`.

Pay attention to the two separate clocks, and indicate the clock-domain boundary with a line through your diagram.

Show your drawing to an assistant.

Checklist

As a final reference for your block-diagram artistry, check whether your future drawings satisfy the following checklist:

- Are clock and reset easily identifiable?
- If you omitted the clock/reset wires, is it clear which clock/reset the pins in your design are connected to?
- Are internal signals properly labeled?
- For each block in the design, can you identify the entity name (e.g. `crc_lfsr`)?
- For each block in the design, can you identify the instance name (e.g. `i_lfsr`)?
- Black boxes:
  - Is there a separate diagram detailing the inner workings, or a reference to more information?
  - Combinational clouds: Is there a description or a reference to code?
  - FSMs: Is there a description, a separate state graph, or a reference to code?
- Ports:
  - Can you easily tell which are inputs and outputs?
- Complex buses:
  - Can you tell which communication protocol is used?
  - Are all protocol parameters specified (e.g. data width)?
  - Can you identify sender/manager/initiator and receiver/subordinate/target?
- Multiple clocks:
  - Is the boundary between clock domains clearly identifiable?
  - Are all signals that pass from one domain into another easily identifiable?

You are done with the exercise.

Discuss your experience with assistants and colleagues.

Cleanup

Student Task - Final Cleanup

Only for ETH users:

- If you followed the instruction at this point you have installed the exercise files in a directory on the workstation's local hard disk (e.g. somewhere in `/scratch/$USER`). By default there is no limit to the amount of local disk space that you can use. Please note that the Tardis workstations are shared among several courses and have limited disk space. Therefore, please make sure to cleanup after yourself once the exercise is done. This is particularly important in the case of exercises that generate large files (e.g. long simulation logs, large netlists, and other files generated by the backend tools).

The VLSI pages are part of the open source VLSI design course offered by the Integrated Systems Laboratory (https://iis.ee.ethz.ch) of ETH Zürich (https://www.ethz.ch), by Luca Benini (https://ee.ethz.ch/the-department/people-a-z/person-detail.luca-benini.html) and Frank K. Gürkaynak (https://iis-people.ee.ethz.ch/~kgf/). See full list of contributors. Unless otherwise noted content available under Creative Commons BY (https://creativecommons.org/licenses/by/4.0/).